

Information Retrieval #8 — Vector Space Model

Severin Mills

24.04.2026

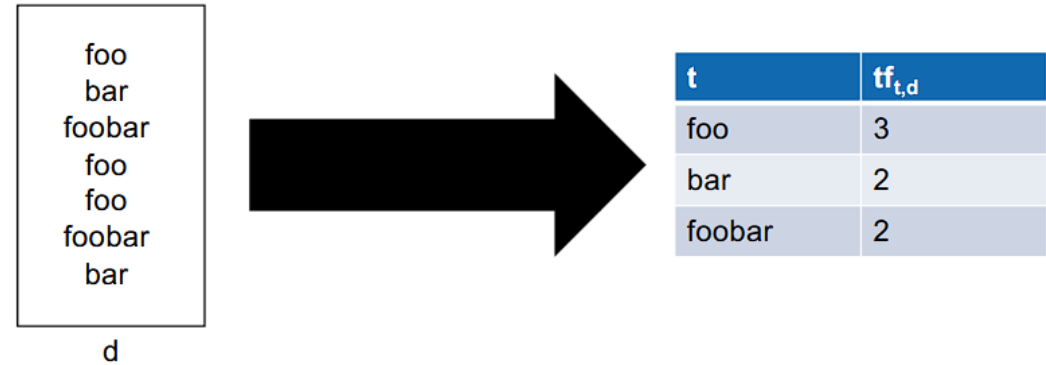


Inhaltsverzeichnis/Agenda

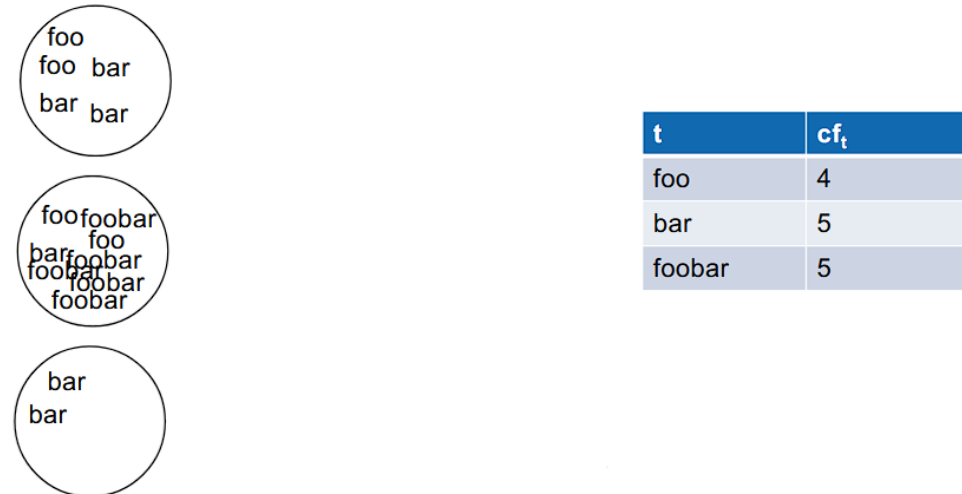
1. Lecture Recap
2. Exercise 7: Vector Space Model (BONUS)
3. Programming
4. Exam Questions
5. Kahoot

Recap: Weights

- Term Frequency:
 - Occurrence of term in document

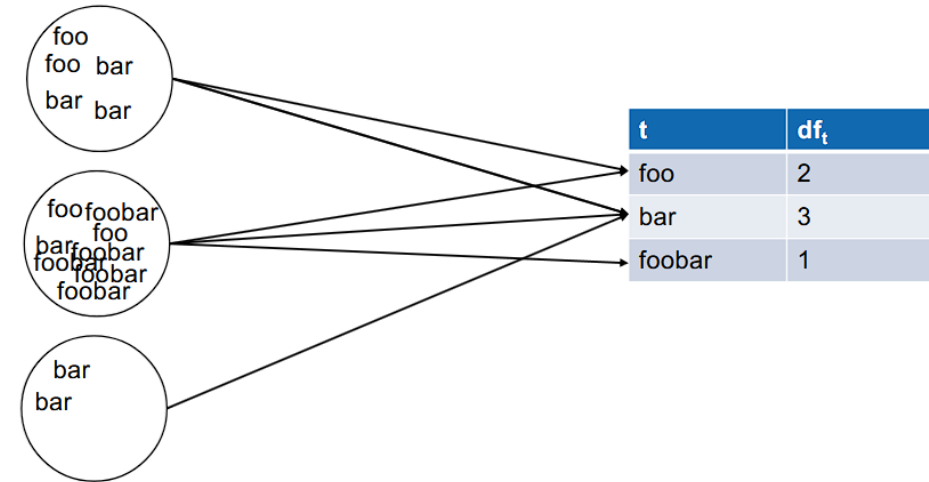


- Collection frequency:
 - Occurrence of term over whole collection

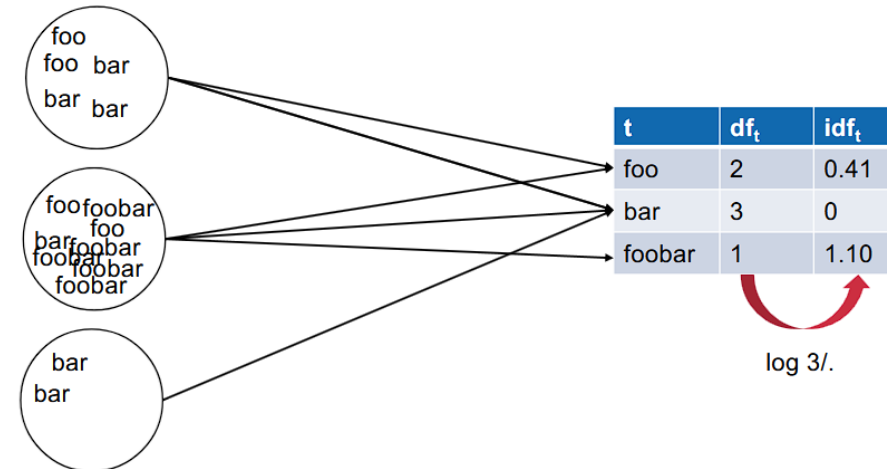


Recap: Weights

- Document Frequency:
 - In how many documents does the term appear?



- Inverse Document Frequency:
 - $idf_t = \log \frac{N}{df_t}$, where N: number of docs



Recap: tf-idf

tf	A	B
foo	5	1
bar	0	4
foobar	2	1



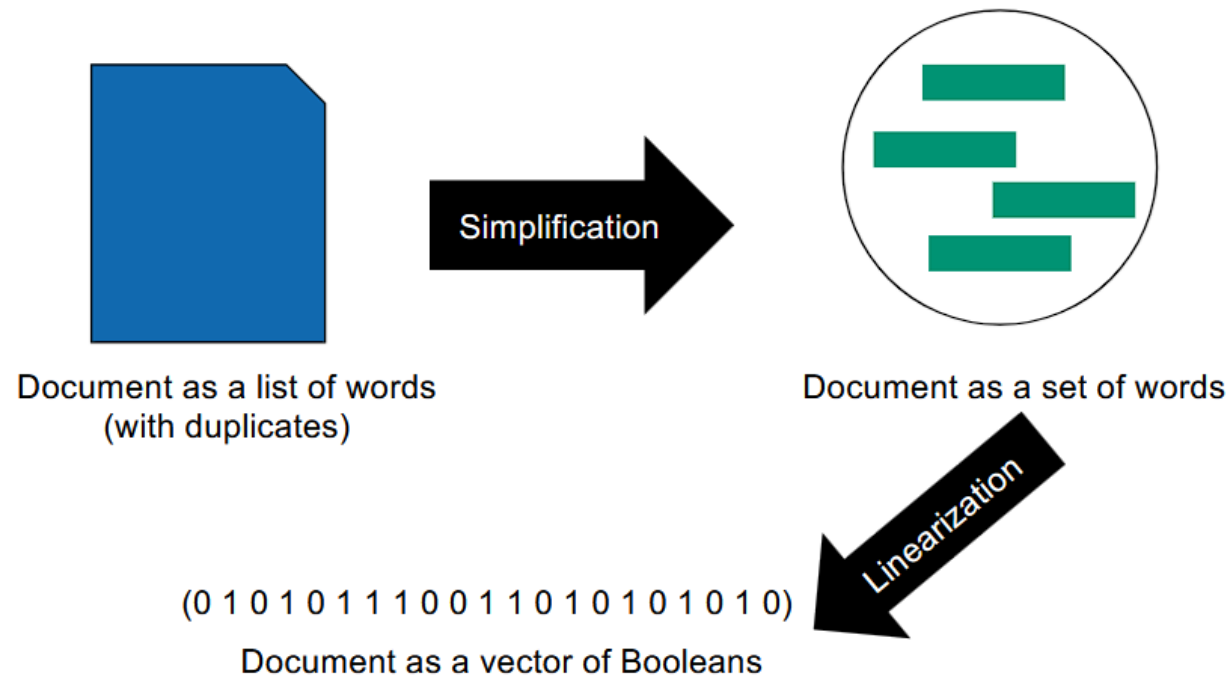
tf-idf	A	B
foo	25	5
bar	0	40
foobar	6	3



idf	
foo	5
bar	10
foobar	3

Vector Space Model

- Core Idea: Represent documents as vectors

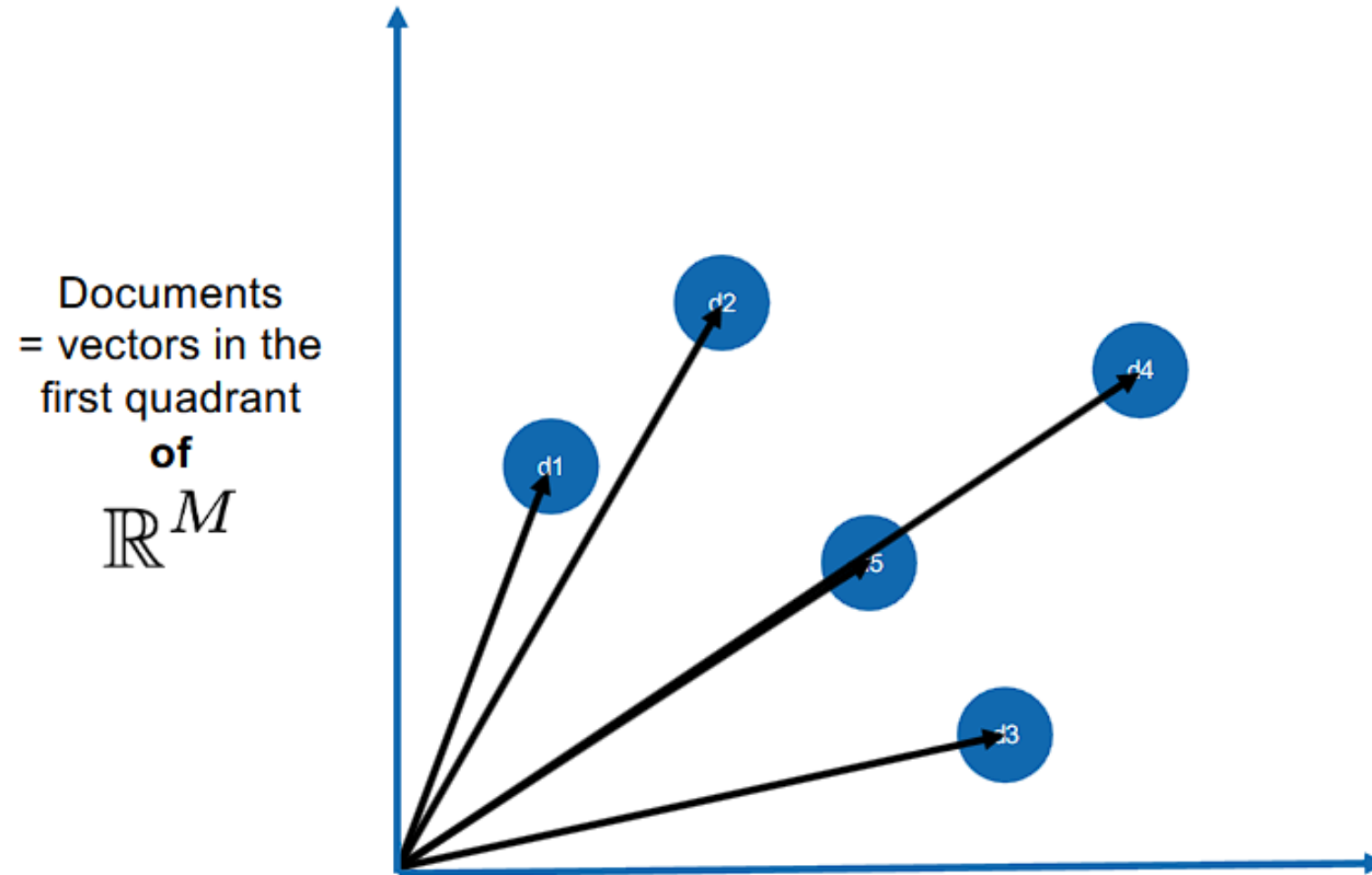


Boolean Vector

- Boolean entries: not very useful for ranking
 - No way to argue about occurrence, zones, etc.
- Better: use positive real numbers
 - e.g. tf-idf, term frequency, etc.

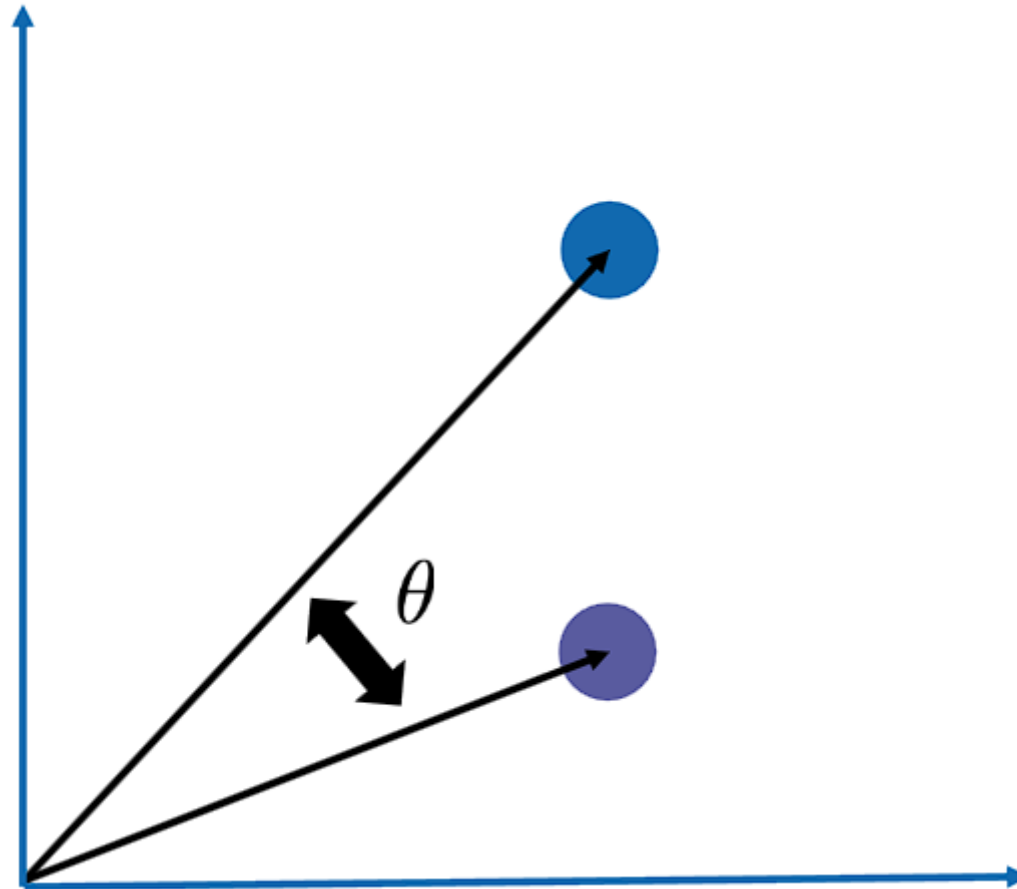
$$\begin{pmatrix} 1.5 \\ 1 \\ 3 \\ 5 \\ 0 \\ 0 \\ 10.4 \\ 0 \end{pmatrix}$$

Vectorization



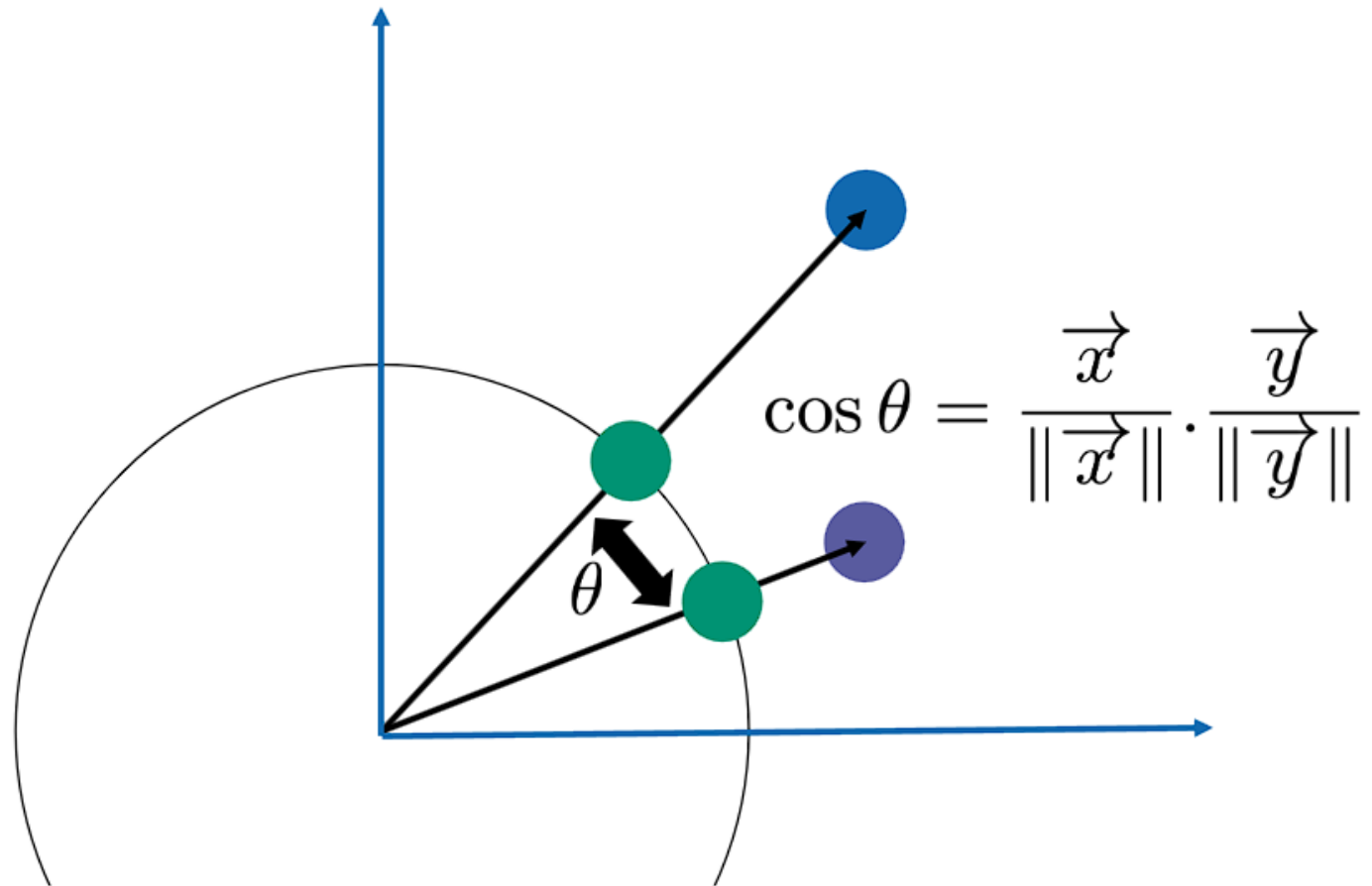
Measuring similarity

- Why is this not optimal?
 - No normalization: two similar documents with different vector lengths would not be very similar according to this calculation

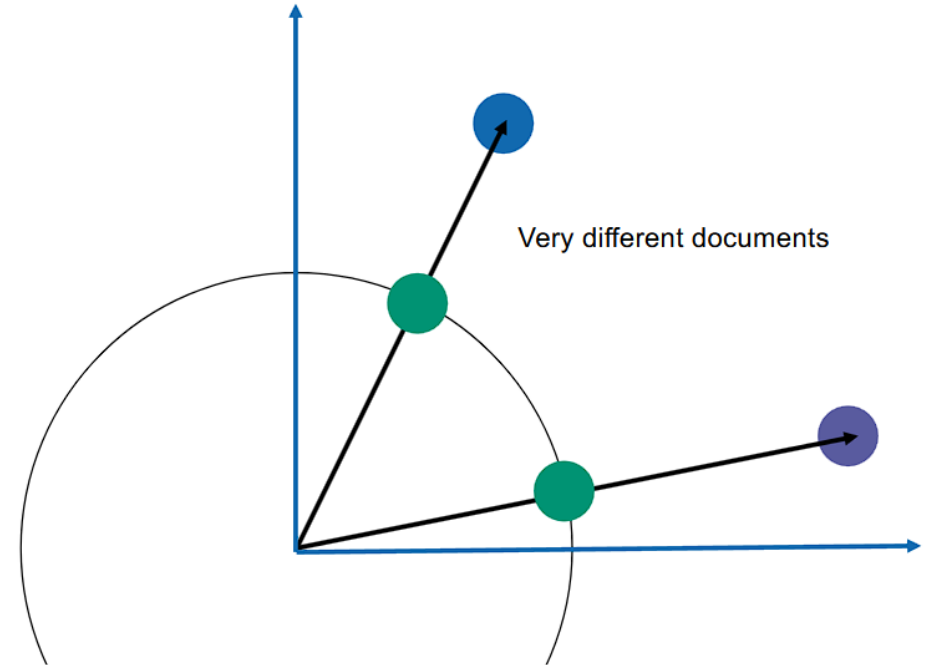
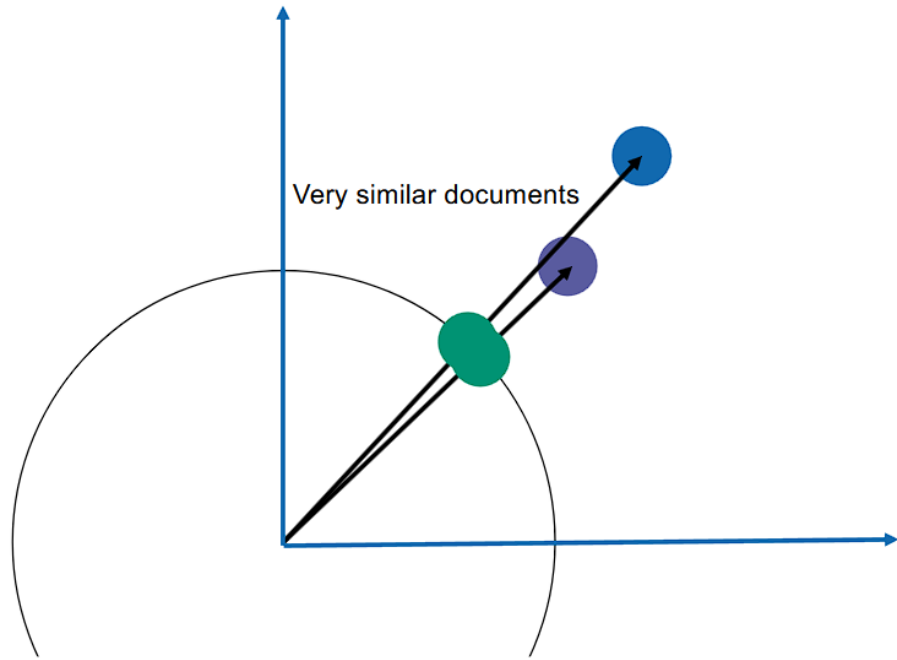


Measuring similarity

- Cosine similarity
 - + Normalization

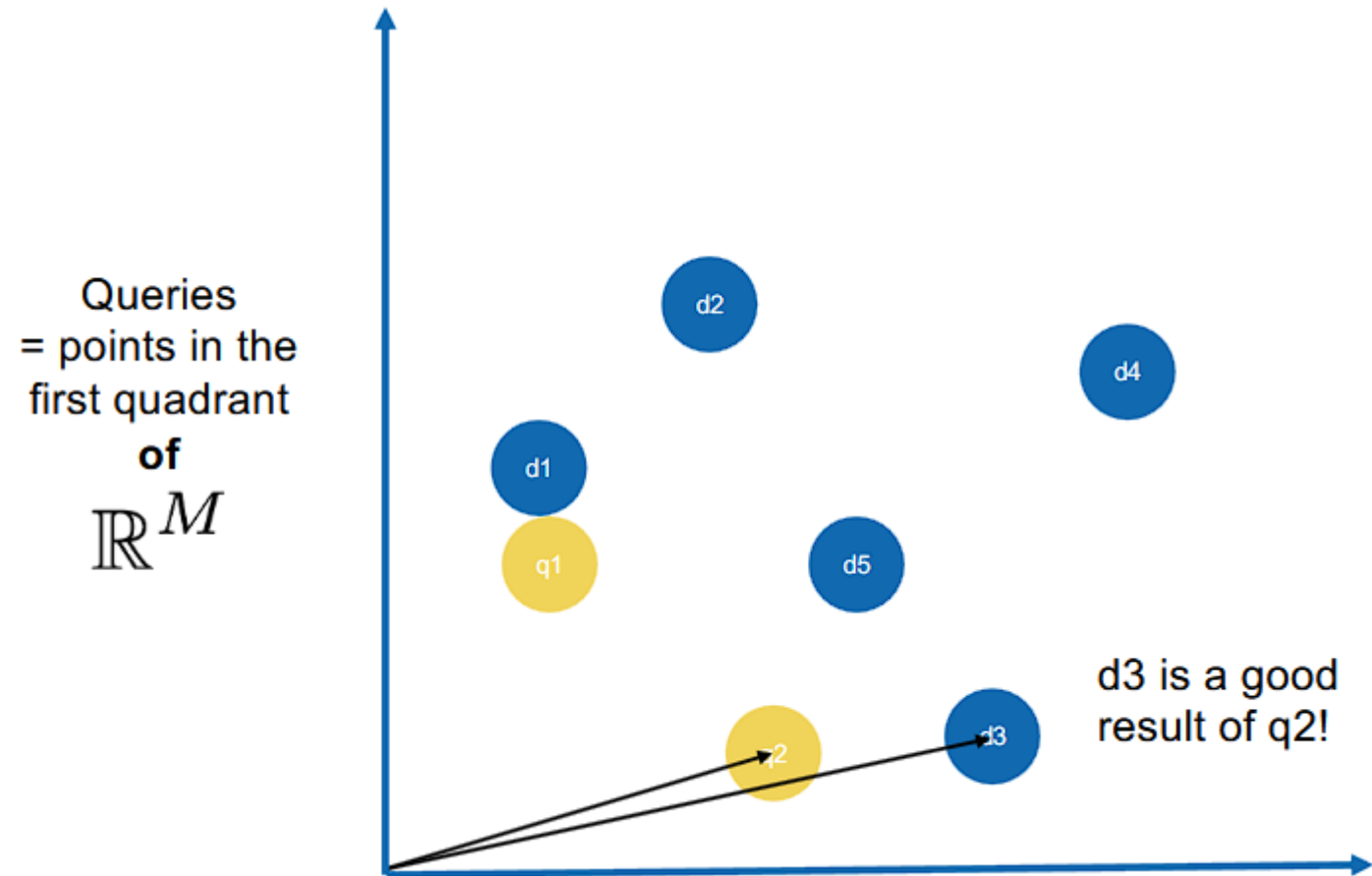


Measuring similarity



Querying on the VSM

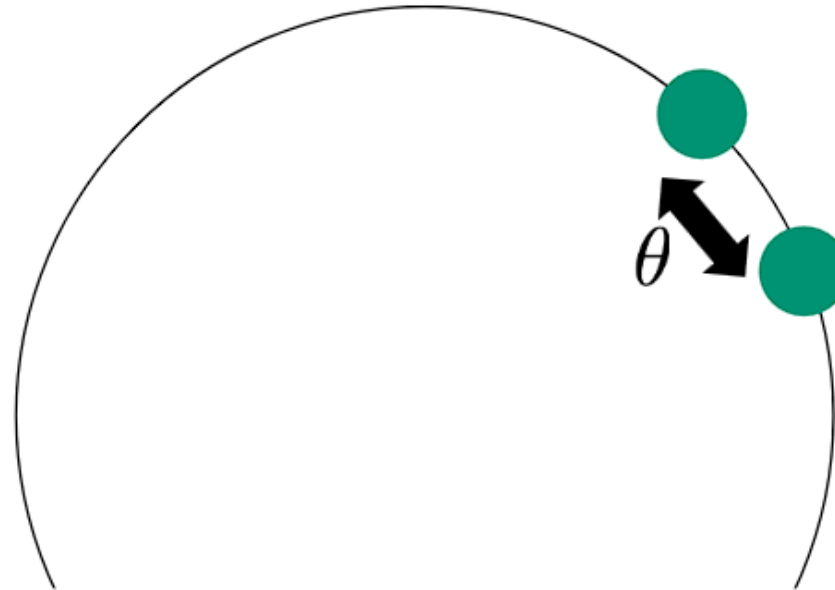
- Queries are also documents!



Querying on the VSM

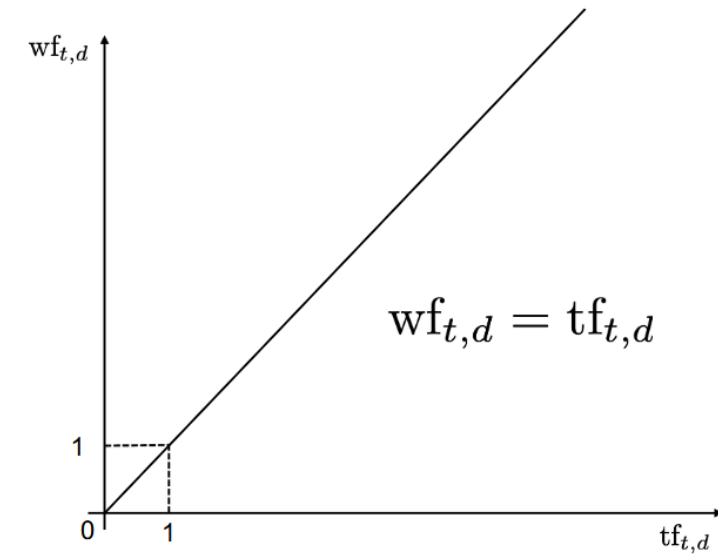
- Calculate similarity between query and documents
- Since query size is constant, it stays efficient

$$\text{sim}(\vec{d}, \vec{q}) = \sum_t \frac{\text{tf-idf}_{td} \times \text{tf-idf}_{tq}}{\|\vec{q}\| \times \|\vec{d}\|}$$

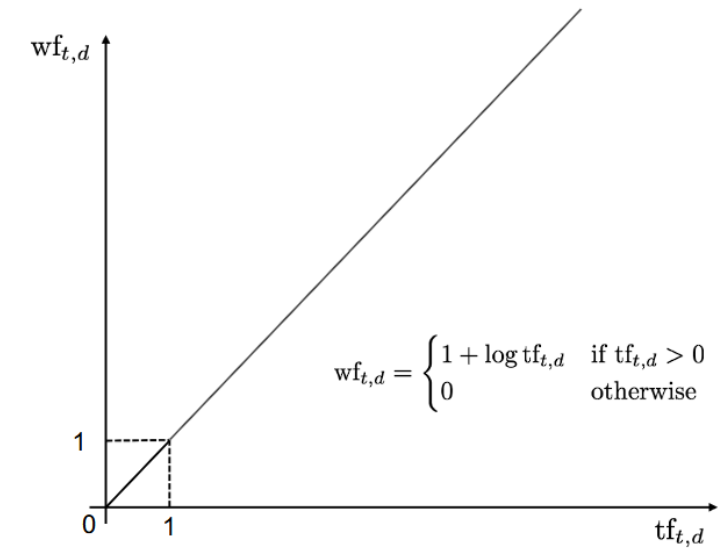


Alternative Weights

- Natural term frequency scaling:

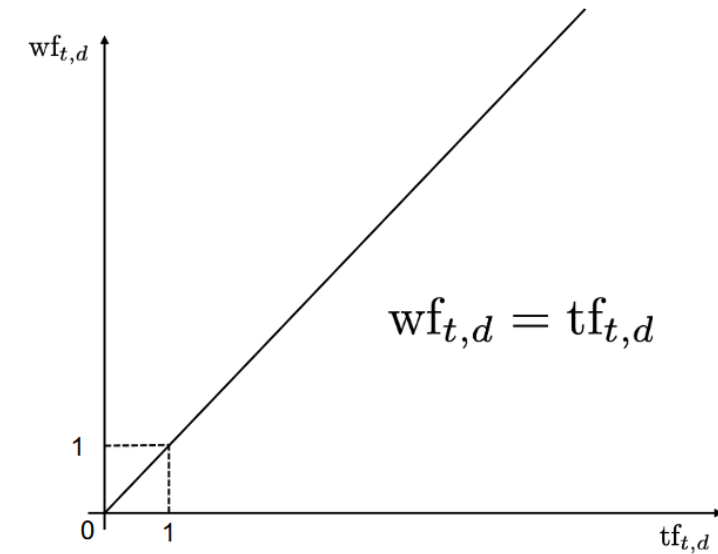


- Sublinear term frequency scaling:

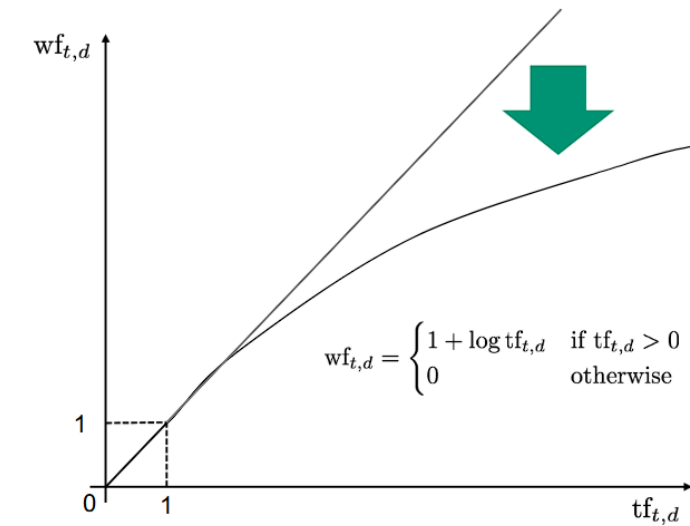


Alternative Weights

- Natural term frequency scaling:

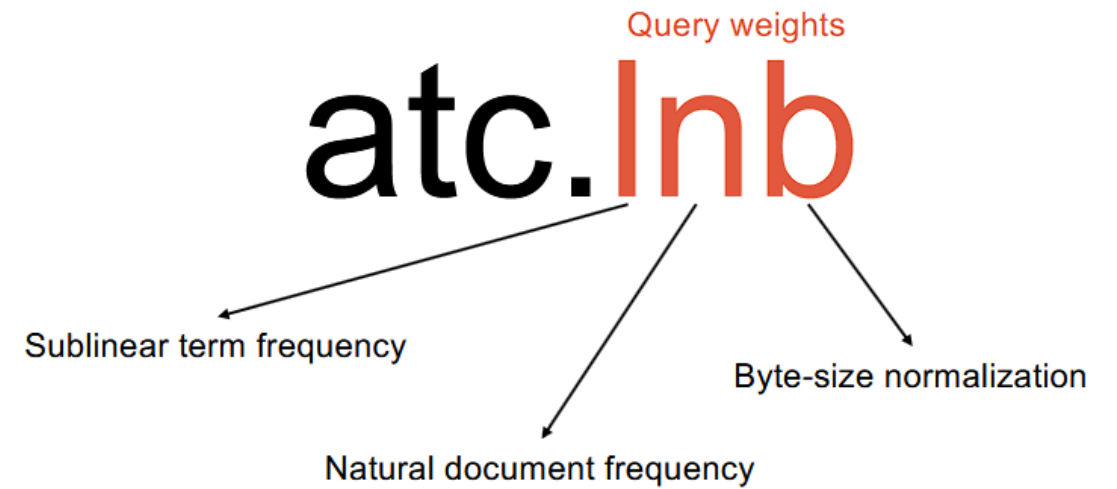
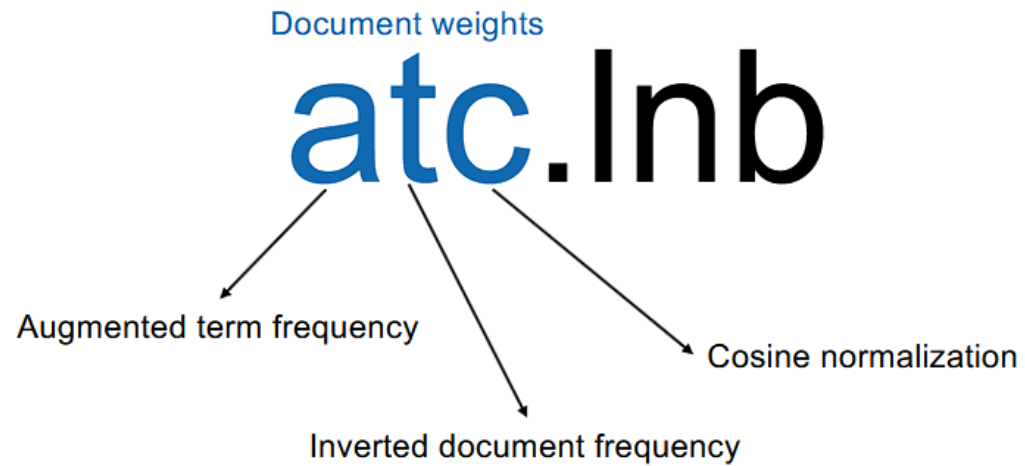


- Sublinear term frequency scaling:



... and many more!

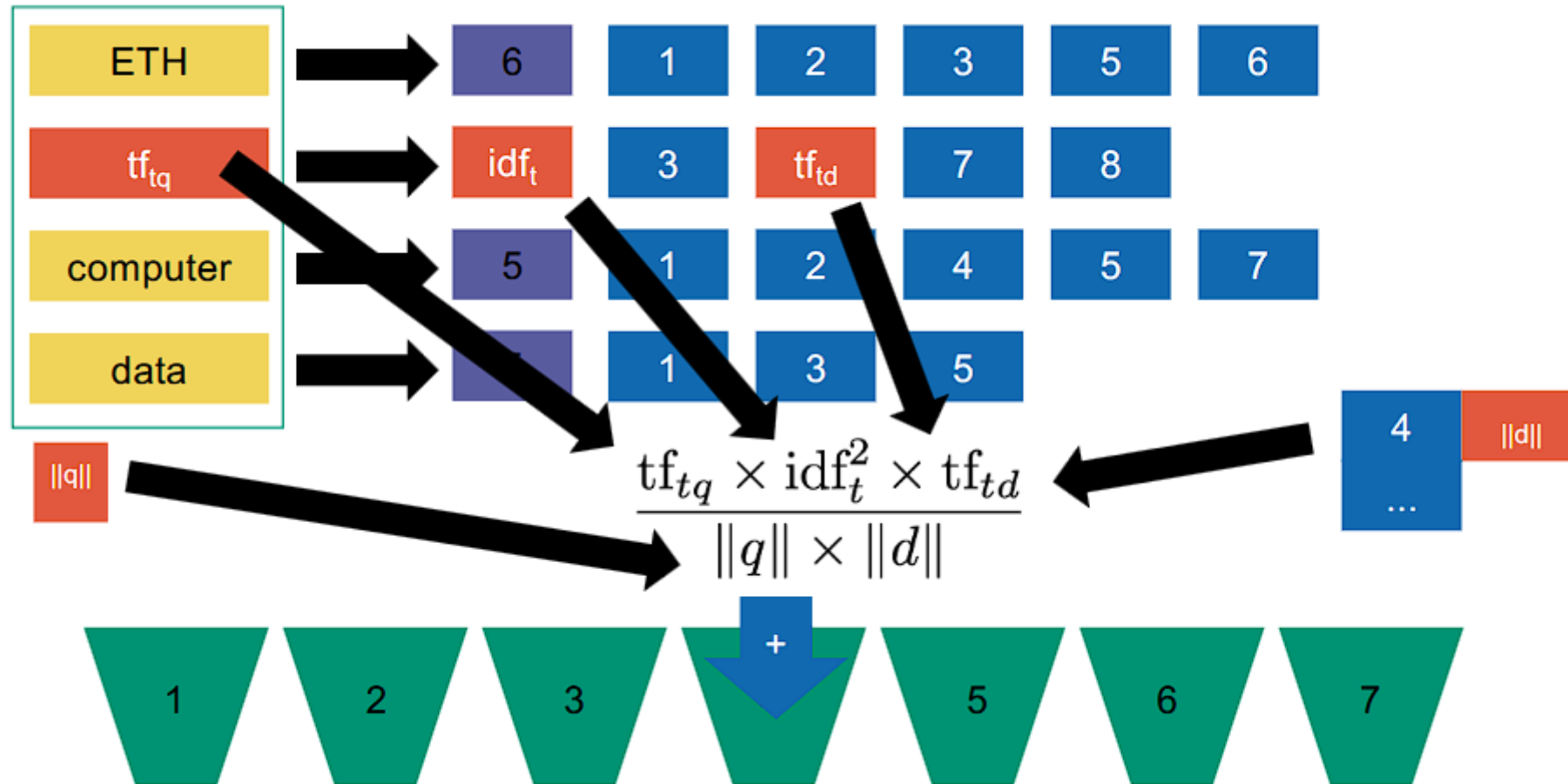
SMART Notation



SMART Notation

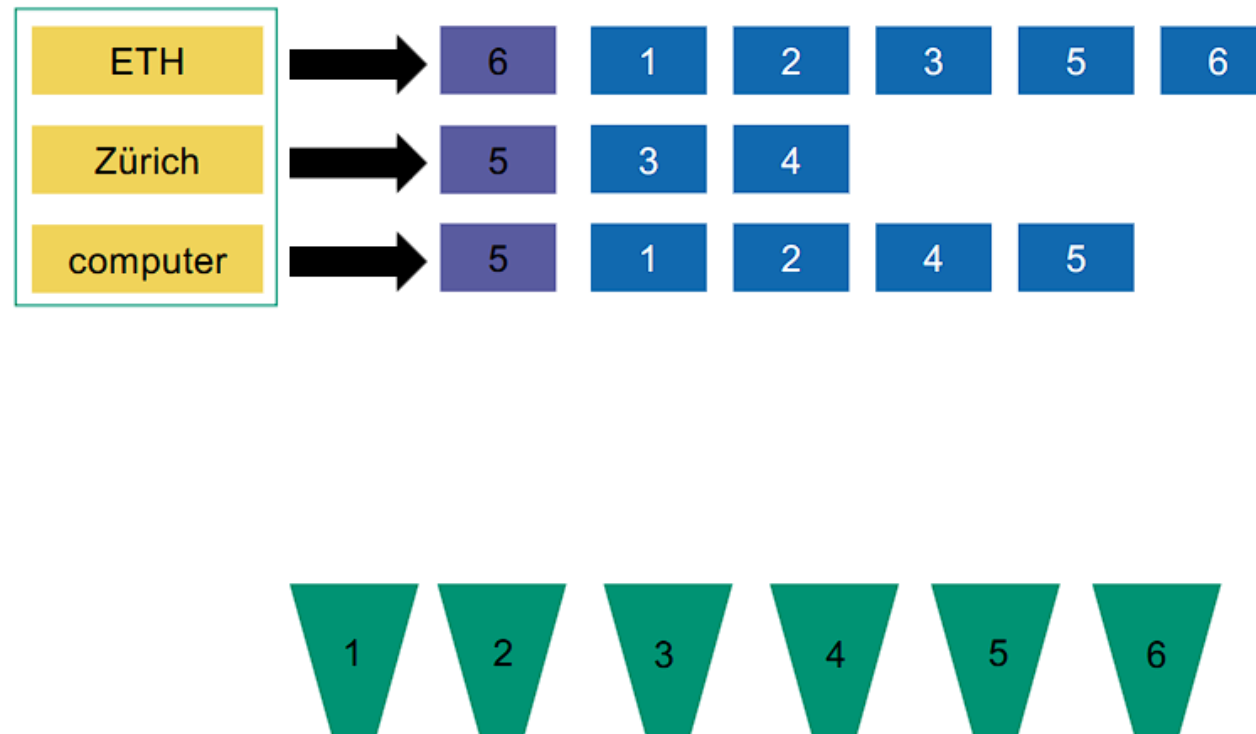
Term frequency		Document frequency		Normalization	
n (natural)	$tf_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(tf_{t,d})$	t (idf)	$\log \frac{N}{df_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2 + w_2^2 + \dots + w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times tf_{t,d}}{\max_t(tf_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N - df_t}{df_t}\}$	u (pivoted unique)	$1/u$ (Section 6.4.4)
b (boolean)	$\begin{cases} 1 & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^\alpha, \alpha < 1$
L (log ave)	$\frac{1 + \log(tf_{t,d})}{1 + \log(\text{ave}_{t \in d}(tf_{t,d}))}$				

Adapting the Inverted Index



Adapting the Inverted Index

- Term-at-a-time vs. Document-at-a-time



Adapting the Inverted Index



Inexact Top-K Retrieval

- Postings lists can be very large

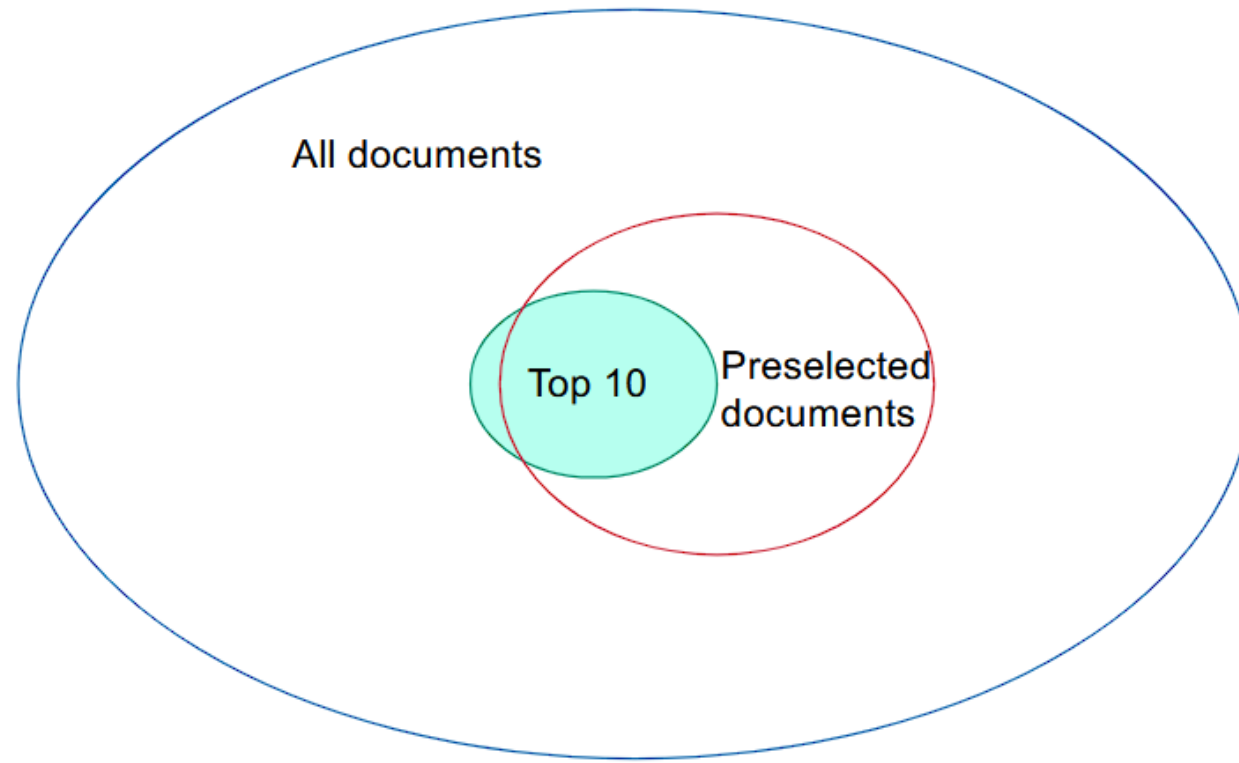


Many accumulators to maintain
Many computations



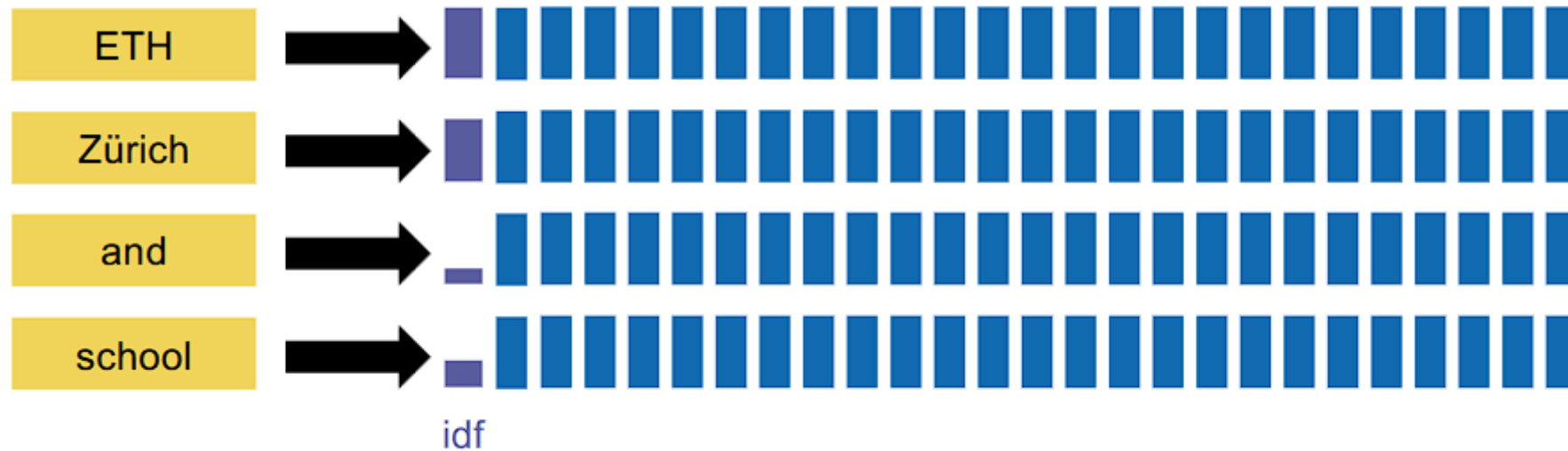
Inexact Top-K Retrieval

- Preselect a document set



Index Elimination

- Eliminate frequent terms (low idf), since most likely stop words



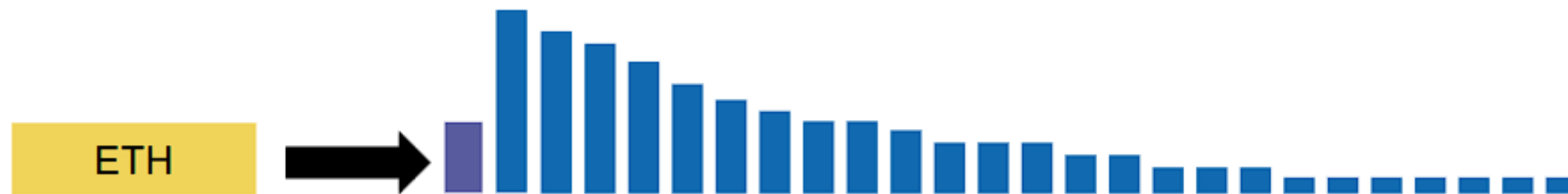
Index Elimination

- Alternate approach, but might eliminate too many documents



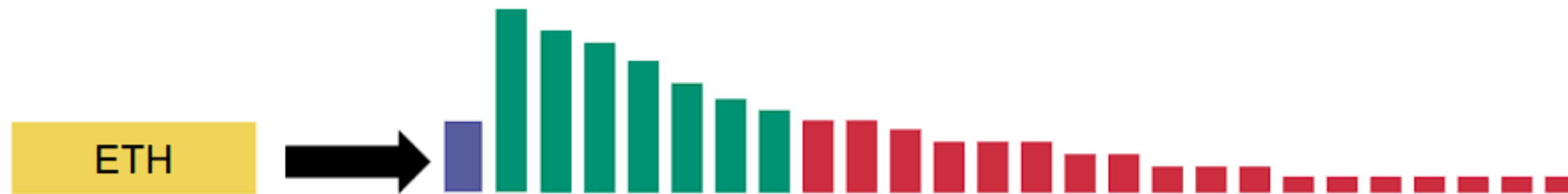
Champion Lists

1. **Sort by term frequency**
2. Only keep top r documents
3. Union top r documents from each term



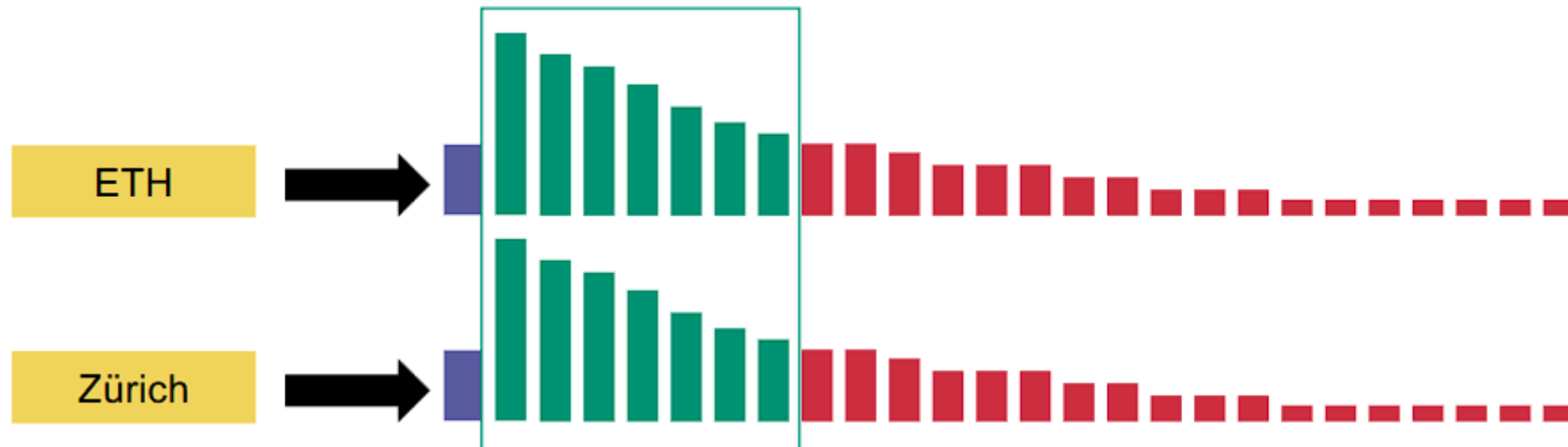
Champion Lists

1. Sort by term frequency
- 2. Only keep top r documents**
3. Union top r documents from each term



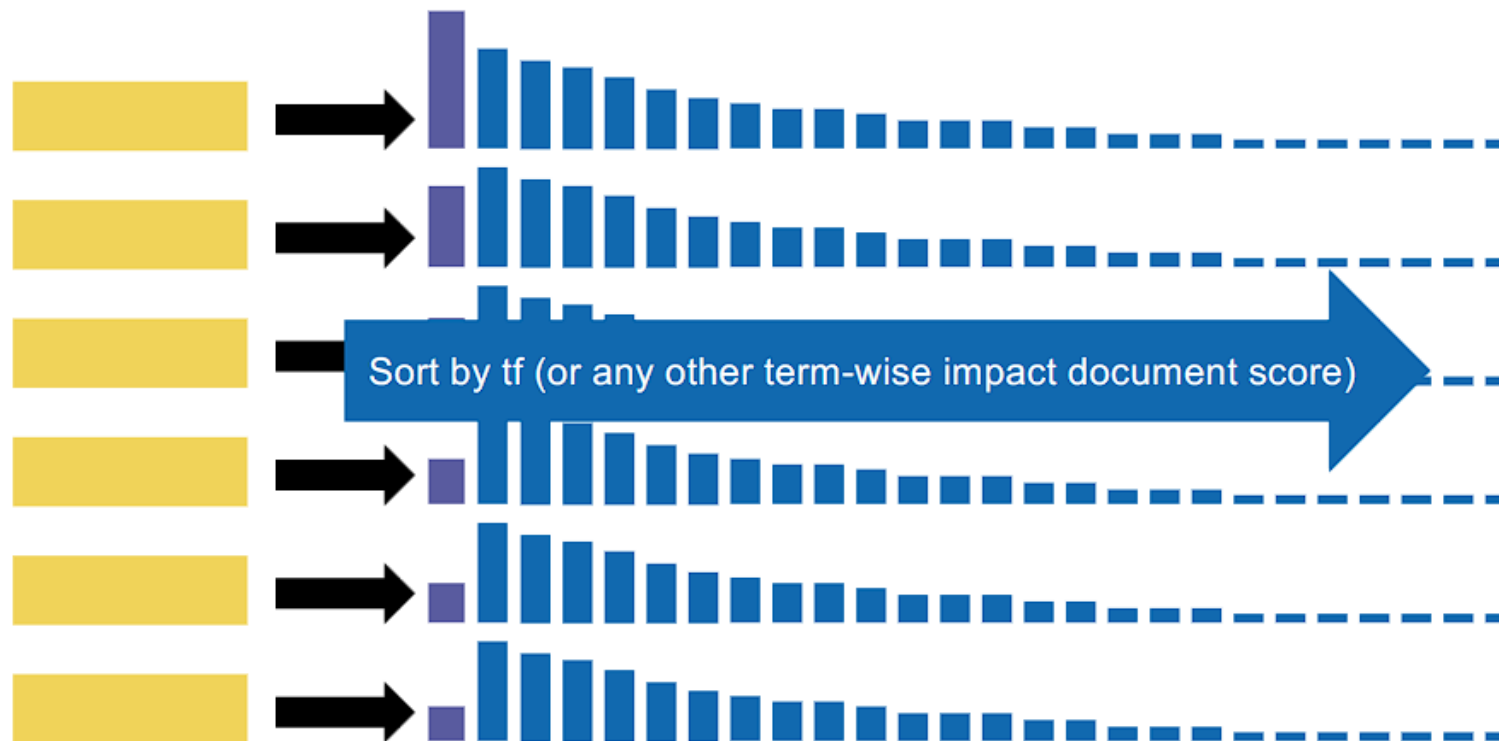
Champion Lists

1. Sort by term frequency
2. Only keep top r documents
3. **Union top r documents from each term**



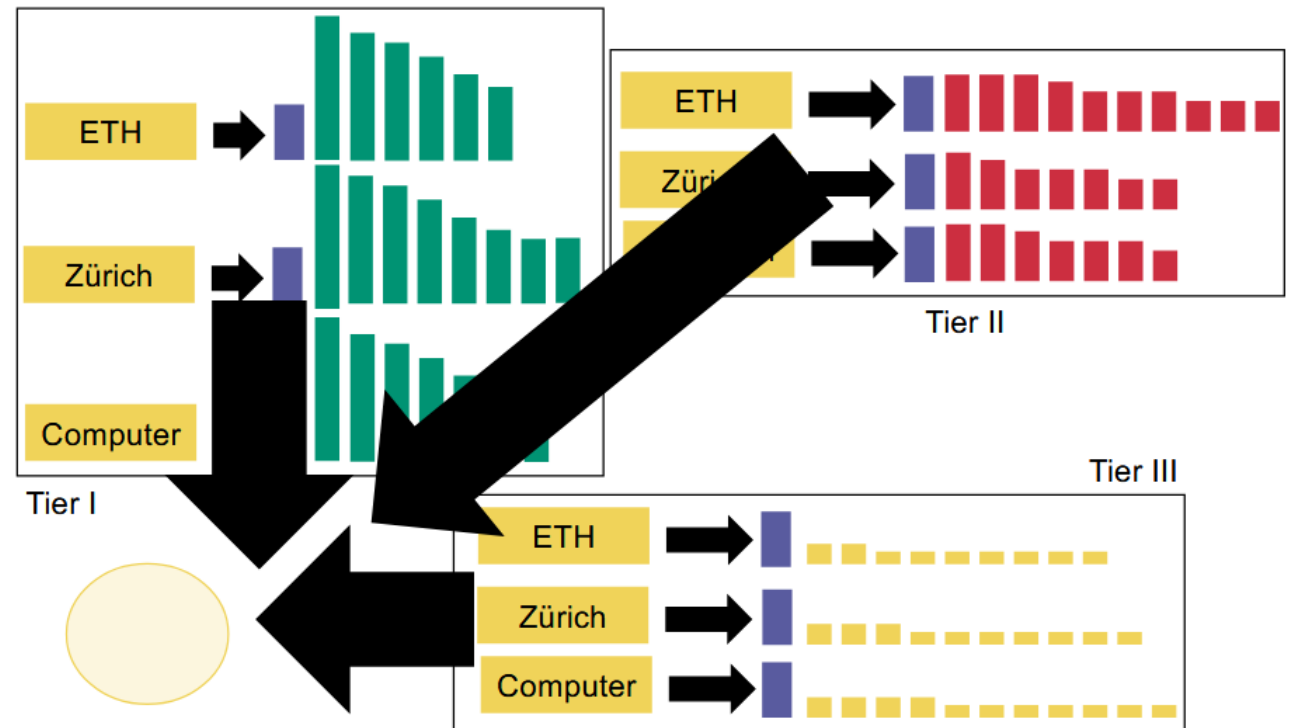
Impact ordering

- Sort by term frequency and inverted document frequency, use per-term Champion Lists
 - Start top left, traverse term-at-a-time
 - What if we don't have enough results?



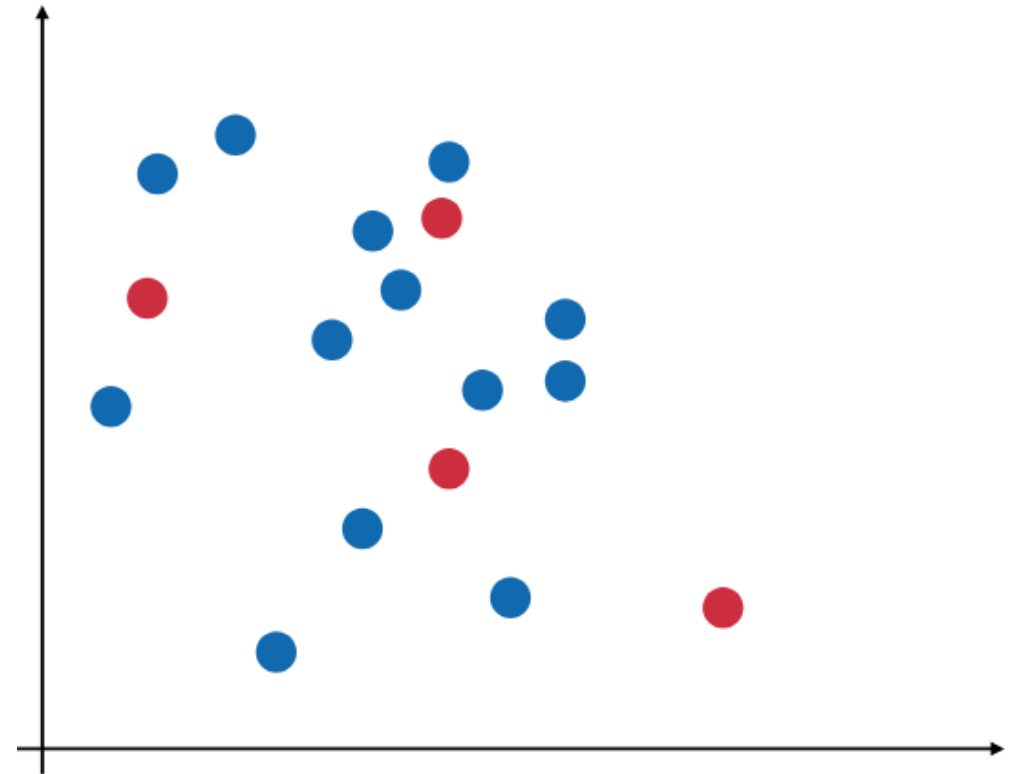
Tiered Indices

1. Grab documents from first tier
2. If not enough, descend tier and grab documents
3. And so on...



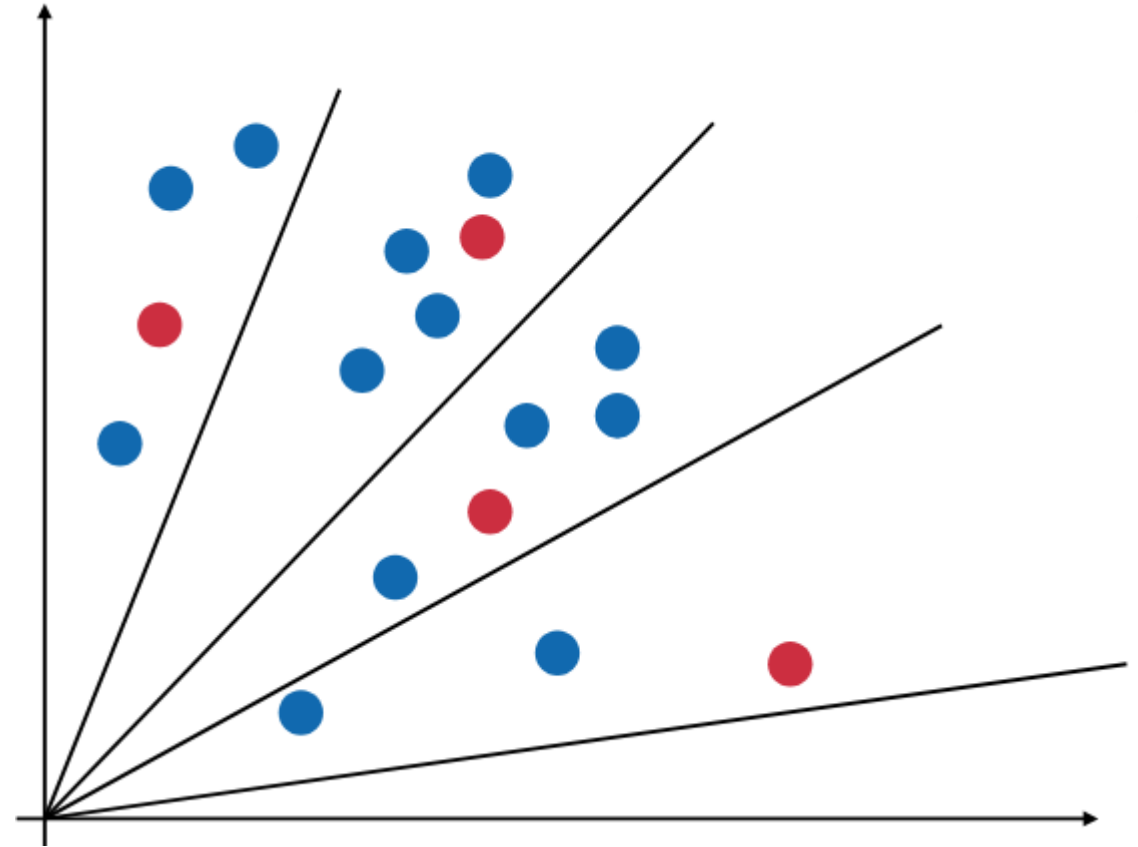
Clustering

1. **Randomly pick leaders in VSM ($\sqrt{N}$)**
2. Cluster document spaces (nearest leader)
3. Pick leader closest to query
4. Compute scores only for local cluster



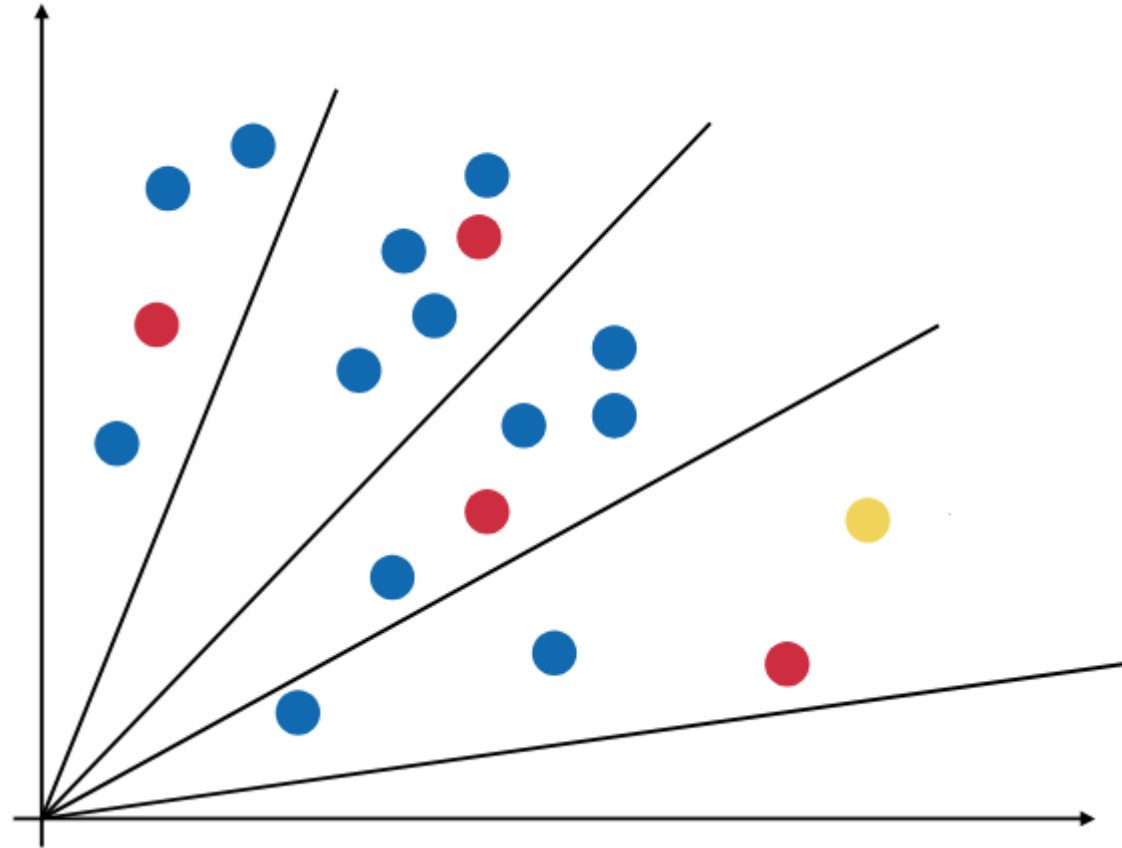
Clustering

1. Randomly pick leaders in VSM ($\sqrt{N}$)
2. **Cluster document spaces (nearest leader)**
3. Pick leader closest to query
4. Compute scores only for local cluster



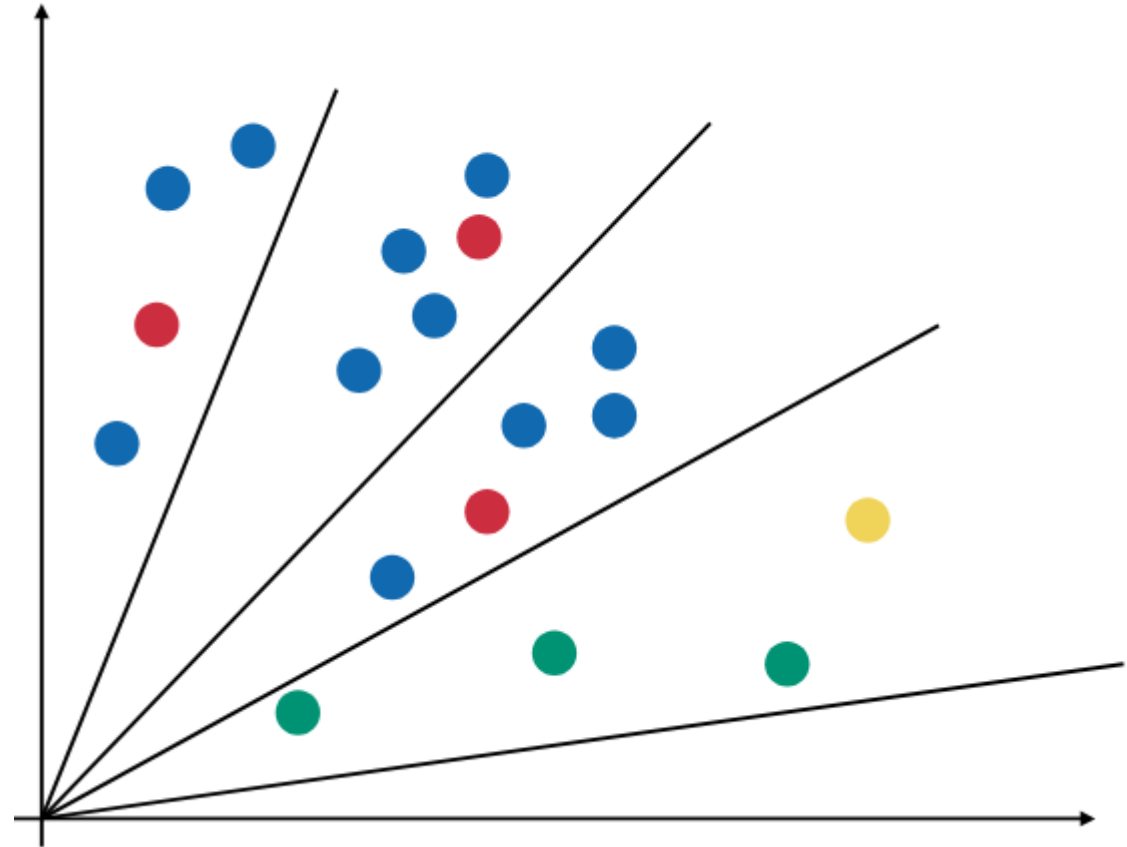
Clustering

1. Randomly pick leaders in VSM ($\sqrt{N}$)
2. Cluster document spaces (nearest leader)
- 3. Pick leader closest to query**
4. Compute scores only for local cluster



Clustering

1. Randomly pick leaders in VSM ($\sqrt{N}$)
2. Cluster document spaces (nearest leader)
3. Pick leader closest to query
4. **Compute scores only for local cluster**



Exercise 7: Vector Space Model (Bonus Quiz)

- Same concept as previously
- Graded Moodle quiz starts Friday, April 24th, 15:00
- Graded Moodle ends Friday, May 1st, 15:00
- Minimum marks to pass: 3 out of 6

Exercise 7: Vector Space Model (Bonus Quiz)

Programming:

- Ranked retrieval system based on vector space model
- Can handle free form text queries
- Your part:
 - Build the index
 - Implement helper function calculating term frequency, document frequency, inverse document frequency, tf-idf weight and cosine normalization
 - Build scoring function

Exercise 7: Vector Space Model (Bonus Quiz)

Build the index

- For each term we need to know:
 1. Number of documents in which the term appears
 2. Number of times a term appears for each of the documents that contain the term
- Up to you how to build such a data structure

Implement helper functions

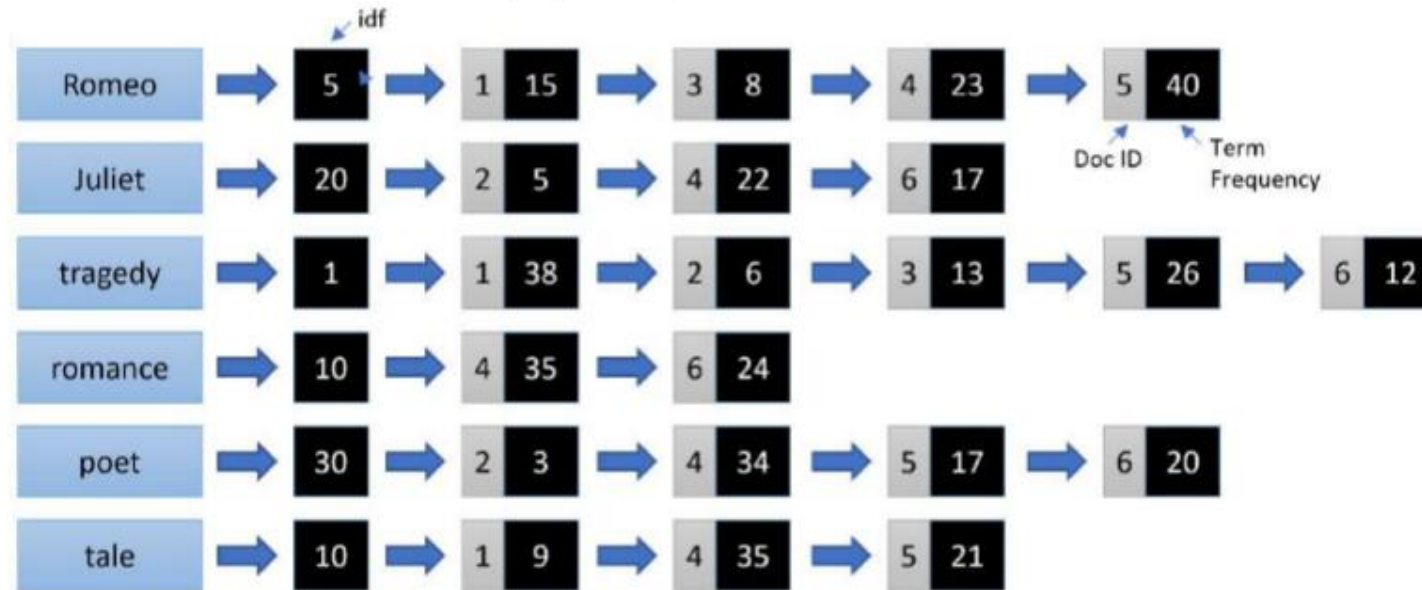
- Implement `tf`, `df`, `idf`, `tf_idf`, `norm_cosine`
- Implement scoring function
 - Compute scores, normalize, return top-K

Exam Questions

The index remains the same for the next few questions and will be repeated on each screen.

For the sake of keeping computations reasonable without a calculator, we assume a setup corresponding to the SMART notation **ntn.bnn**. This means that:

- we use **tf-idf**'s in **document vectors**;
- query terms are **not repeated** and we use a **factor of 1** instead of idf's in the **query vectors**;
- we **do not renormalize** document nor query vectors;



Compute the score of **document 2** against the following free-text query given the index.

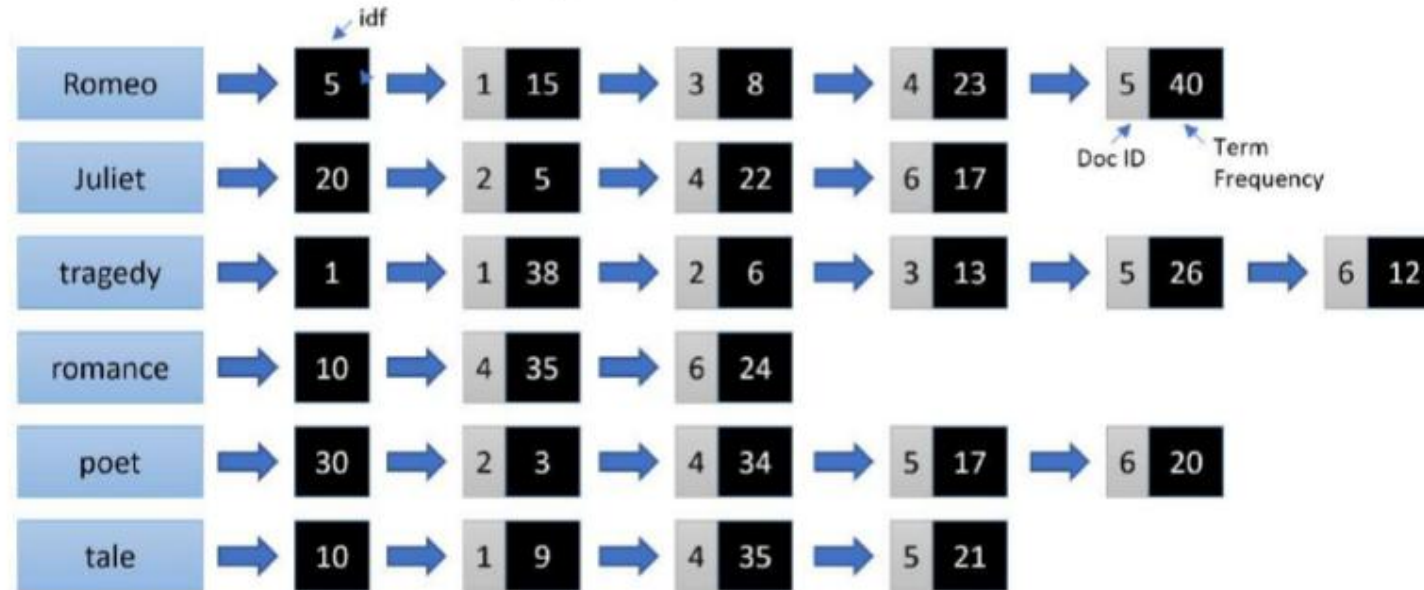
Query: poet

Exam Questions

The index remains the same for the next few questions and will be repeated on each screen.

For the sake of keeping computations reasonable without a calculator, we assume a setup corresponding to the SMART notation **ntn.bnn**. This means that:

- we use **tf-idf**'s in **document vectors**;
- query terms are **not repeated** and we use a **factor of 1** instead of idf's in the **query vectors**;
- we **do not renormalize** document nor query vectors;



Compute the score of **document 2** against the following free-text query given the index.

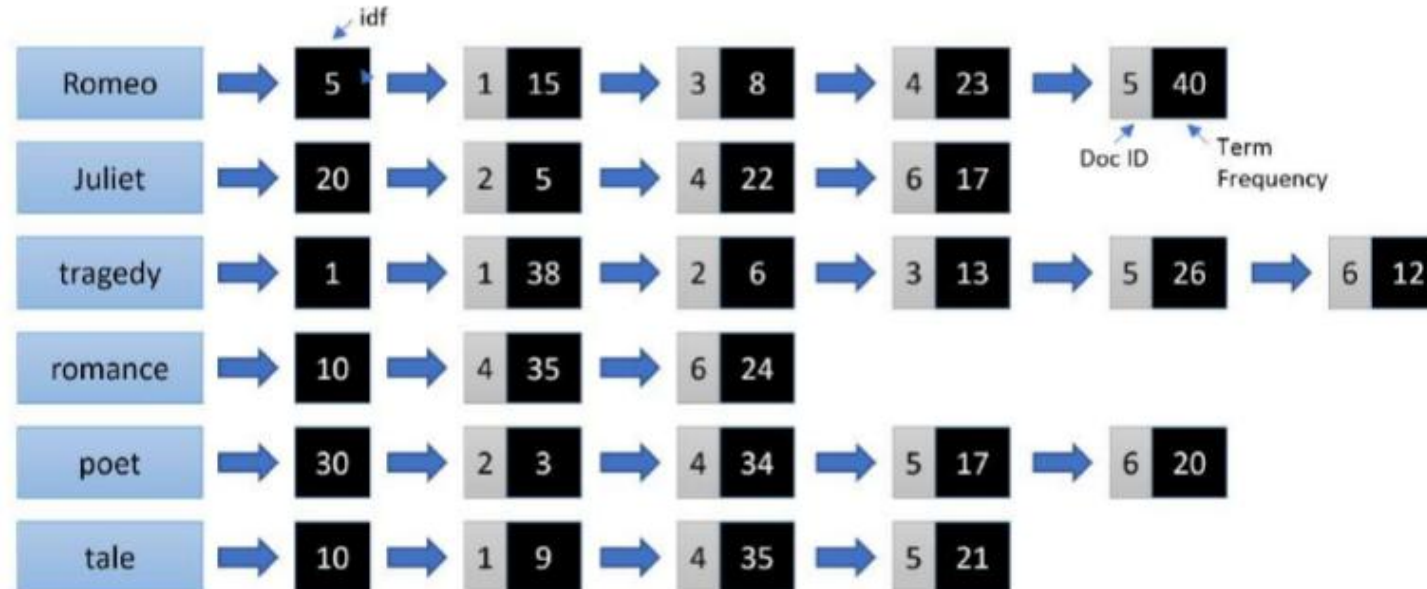
Query: poet

Solution: $30 \cdot 3 = 90$

Exam Questions

For the sake of keeping computations reasonable without a calculator, we assume a setup corresponding to the SMART notation **ntn.bnn**. This means that:

- we use **tf-idf**'s in **document vectors**;
- query terms are **not repeated** and we use a **factor of 1** instead of idf's in the **query vectors**;
- we **do not renormalize** document nor query vectors;



Compute the score of **document 1** against the following free-text query given the index.

Query: **Romeo tragedy tale**

Exam Questions

Solution: $5 \cdot 15 + 1 \cdot 35 + 10 \cdot 9 = 203$

Solution: $5 \cdot 15 + 1 \cdot 35 + 10 \cdot 9 = 203$

Exam Questions

- ☐ Using raw term frequencies in scores generally suffers from a critical problem: all terms are considered equally important when it comes to assessing relevancy on a query
- ☐ We can use the outer product to quantify the similarity between two documents in the vector space.
- ☐ In the formula of the inverse document frequency for some term t , N denotes the number of documents in a collection which contains term t .
- ☐ Term-at-a-time scoring and document-at-a-time scoring both produce the same scores.
- ☐ The SMART notation takes the form $ddd.qqq$ where the first triplet gives the term weighting of the document vector, while the second triplet gives the weighting in the query vector.

Exam Questions

- ✓ Using raw term frequencies in scores generally suffers from a critical problem: all terms are considered equally important when it comes to assessing relevancy on a query
- ❑ We can use the outer product to quantify the similarity between two documents in the vector space.
- ❑ In the formula of the inverse document frequency for some term t , N denotes the number of documents in a collection which contains term t .
- ✓ Term-at-a-time scoring and document-at-a-time scoring both produce the same scores.
- ✓ The SMART notation takes the form $ddd.qqq$ where the first triplet gives the term weighting of the document vector, while the second triplet gives the weighting in the query vector.

Exam Questions

is one heuristic used for inexact top K document retrieval. In this heuristic, we only consider documents containing terms whose exceeds a preset threshold. One way of viewing this heuristic is the terms are treated as stop words and do not contribute to scoring.

- Index elimination
- Champion list
- Static query score
- Impact ordering
- Inverted Document Frequency
- Term Frequency
- Collection Frequency
- Low
- High

Exam Questions

Index elimination is one heuristic used for inexact top K document retrieval. In this heuristic, we only consider documents containing terms whose exceeds a preset threshold. One way of viewing this heuristic is the terms are treated as stop words and do not contribute to scoring.

- Index elimination
- Champion list
- Static query score
- Impact ordering
- Inverted Document Frequency
- Term Frequency
- Collection Frequency
- Low
- High

Exam Questions

Index elimination is one heuristic used for inexact top K document retrieval. In this heuristic, we only consider documents containing terms whose Inverted Document Frequency exceeds a preset threshold. One way of viewing this heuristic is the terms are treated as stop words and do not contribute to scoring.

- Index elimination
- Champion list
- Static query score
- Impact ordering
- Inverted Document Frequency
- Term Frequency
- Collection Frequency
- Low
- High

Exam Questions

Index elimination is one heuristic used for inexact top K document retrieval. In this heuristic, we only consider documents containing terms whose Inverted Document Frequency exceeds a preset threshold. One way of viewing this heuristic is the Low [] terms are treated as stop words and do not contribute to scoring.

- Index elimination
- Champion list
- Static query score
- Impact ordering
- Inverted Document Frequency
- Term Frequency
- Collection Frequency
- Low
- High

Exam Questions

Index elimination is one heuristic used for inexact top K document retrieval. In this heuristic, we only consider documents containing terms whose Inverted Document Frequency exceeds a preset threshold. One way of viewing this heuristic is the Low Inverted Document Frequency terms are treated as stop words and do not contribute to scoring.

- Index elimination
- Champion list
- Static query score
- Impact ordering
- Inverted Document Frequency
- Term Frequency
- Collection Frequency
- Low
- High

Kahoot

<https://create.kahoot.it/details/duplicate-of-information-retrieval-ex-07-vector-space-models-mschoeb/d790da6e-9dd4-4191-bdd5-e71a97d32ca3?drawer=>